

Simple Linear Regression

Complete Notes with Detailed Step-by-Step Explanation

(Har chhoti se chhoti concept ko asaan bhasha mein samjhaya gaya hai)

1. Introduction — Yeh Topic Kya Hai?

Simple Linear Regression, Machine Learning ke sabse basic aur sabse important algorithms mein se ek hai. Yeh Supervised Machine Learning ke andar aata hai, aur Supervised Machine Learning ke do bade categories hote hain — Regression aur Classification. Simple Linear Regression, Regression wale part ka sabse pehla aur sabse simple algorithm hai.

Supervised ML → Regression — iska matlab hai ki hum apne model ko ek dataset dete hain jisme input (X) aur uska sahi output (Y) pehle se diya hota hai. Model in dono ke beech ka relationship seekhta hai, taaki future mein naye input ke liye woh sahi output predict kar sake.

Regression ka use hum tab karte hain jab hume predict karna ho ek continuous (numeric) value — jaise height, price, temperature, salary, etc. (Isse alag, Classification mein hum categories predict karte hain, jaise 'spam' ya 'not spam'.)

2. Dataset Samajhna — Features aur Labels

Kisi bhi Machine Learning problem ki shuruaat ek Dataset se hoti hai. Dataset mein do tarah ke columns hote hain:

- **Input Feature (X) — Independent Feature:** Jo value hum jaante hain aur jisse hum prediction karenge. Notes ke example mein yeh 'Weight' hai.
- **Output / Label (Y) — Dependent Feature:** Jo value hum predict karna chahte hain. Isse 'dependent' isliye kehte hain kyunki iski value X par depend karti hai. Notes ke example mein yeh 'Height' hai.

Example Dataset (Weight se Height predict karna):

Weight (X)	Height (Y)
74	170 cm
80	180 cm
75	175.5 cm
...	...

Note: Yahan 'm' se hum dataset ke total rows (training examples) ki sankhya denote karte hain.

3. Model Kaise Kaam Karta Hai — Training Flow

Diagram mein dikhaya gaya flow yeh batata hai ki Model ko pehle TRAIN kiya jaata hai — matlab usse Weight aur Height ka existing dataset dikhaya jaata hai, taaki woh dono ke beech ka pattern/relationship seekh le.

Training ke baad, jab hum Model ko ek naya (New) Weight dete hain jiska Height humein nahi pata, toh Model us seekhe hue pattern ke through us naye weight ke liye Height predict kar deta hai.

New Weight → [Model (Trained)] → Predicted Height

Is poore process ka naam hai Simple Linear Regression — kyunki yahan hum sirf ek input feature (Weight) use karke ek output (Height) predict kar rahe hain, aur relationship ek 'straight line' (linear) maana jaata hai.

4. Best Fit Line aur Error

Jab hum saare data points (X, Y) ko ek graph par plot karte hain, toh woh points ek scatter pattern mein dikhte hain — bilkul ek straight line par nahi hote. Hamara goal hai ek aisi straight line dhoondna jo in sab points ke sabse kareeb se guzre. Isi line ko 'Best Fit Line' kehte hain.

Har data point aur us line ke beech ki vertical distance ko Error kehte hain — matlab actual value aur line ne jo predict kiya, unke beech ka farak.

Goal: Best Fit Line woh line hai jiske liye total Error sabse Minimal (kam se kam) ho.

5. Hypothesis Function — $h\theta(x)$

Best Fit Line ko mathematically represent karne ke liye hum ek equation likhte hain, jise Hypothesis Function kehte hain. Yeh bilkul waise hi hai jaise school mein padhi gayi straight line ki equation:

$$y = m x + c$$

Statistics/ML mein isi equation ko thoda different naming ke saath likha jaata hai:

$$y = \beta_0 + \beta_1 x \quad (\text{ya}) \quad h\theta(x) = \theta_0 + \theta_1 x$$

Dono equations bilkul same concept represent karti hain, bas naam alag hain. ML mein hum generally $h\theta(x) = \theta_0 + \theta_1 x$ wala notation use karte hain.

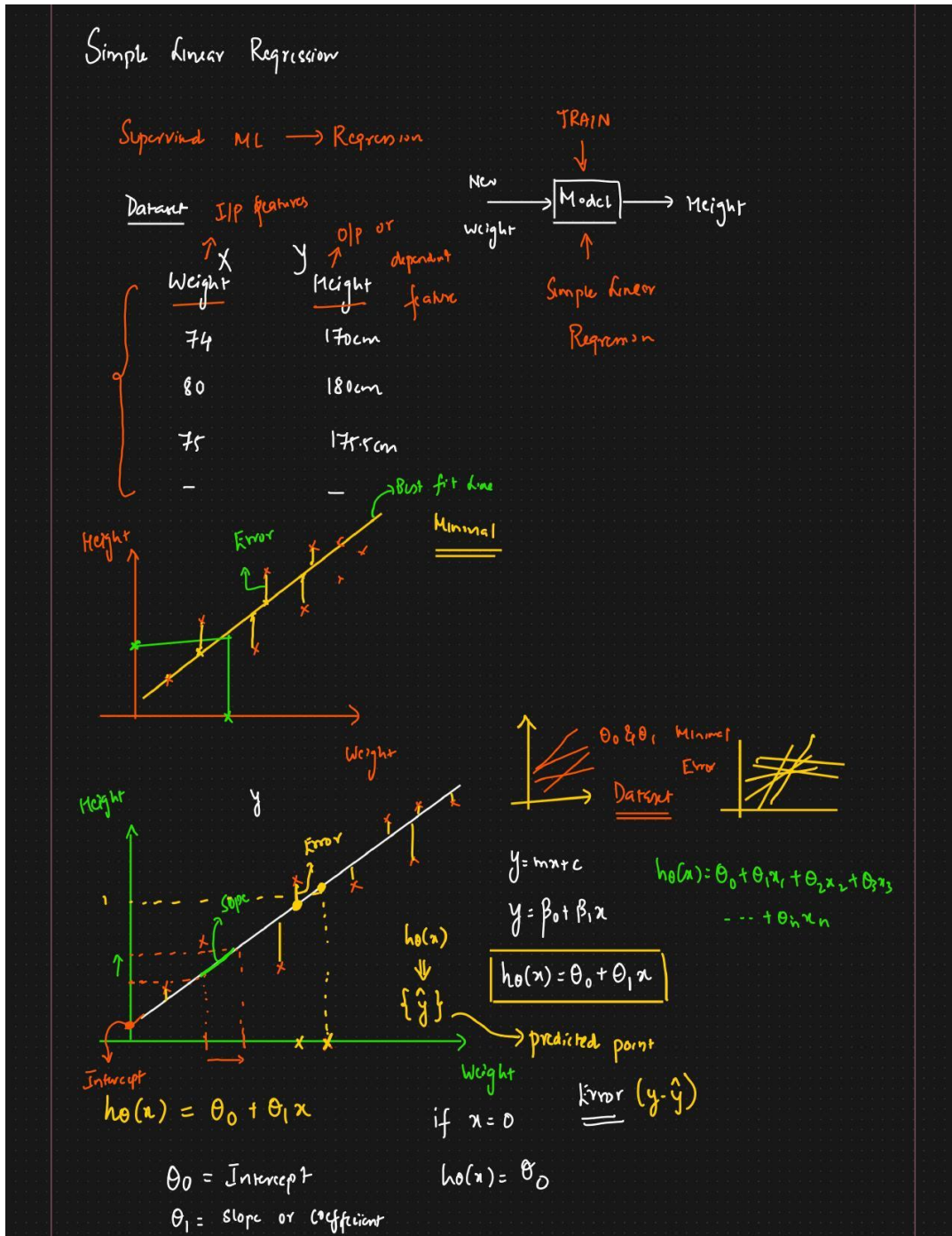
θ_0 aur θ_1 ka Matlab

- θ_0 (theta-zero) = Intercept — yeh woh value hai jahan line, Y-axis ko cross karti hai. Simple shabdon mein, agar $x = 0$ ho, toh $h\theta(x) = \theta_0$ hota hai.
- θ_1 (theta-one) = Slope ya Coefficient — yeh batata hai ki line kitni 'tedhi' (steep) hai, matlab x mein 1 unit ka change hone par y kitna change hoga.

$h\theta(x)$ ka output hi hamara Predicted Point hota hai, jisse hum $\hat{y}$ (y-hat) likhte hain. Error hamesha actual value aur predicted value ke beech ka farak hota hai:

$$\text{Error} = (y - \hat{y})$$

Note: Agar humare paas sirf ek input feature (x) hai, toh use Simple Linear Regression kehte hain. Agar multiple input features ($x_1, x_2, x_3, \dots, x_n$) hon, toh equation extend hoke ban jaati hai: $h\theta(x) = \theta_0 + \theta_1 x_1 + \theta_2 x_2 + \theta_3 x_3 + \dots + \theta_n x_n$ — isse Multiple Linear Regression kehte hain.



Isme dikhaya gaya hai: Dataset, Model training flow, Best Fit Line with Error, aur Hypothesis Function $h_0(x) = \theta_0 + \theta_1 x$ (Intercept & Slope).

6. Cost Function — $J(\theta_0, \theta_1)$

Ab sawaal yeh hai — humein kaise pata chalega ki konsi line 'Best Fit' hai? Iske liye hum ek function banate hain jo batata hai ki hamari line kitni 'achhi' ya kitni 'buri' hai — ise Cost Function kehte hain.

$$J(\theta_0, \theta_1) = (1/2m) \times \sum [h\theta(x^{(i)}) - y^{(i)}]^2 \quad \text{for } i = 1 \text{ to } m$$

Is formula ko step by step samjhte hain:

- $h\theta(x^{(i)}) \rightarrow$ i-th data point ke liye Predicted value.
- $y^{(i)} \rightarrow$ i-th data point ki Actual (true) value.
- $(h\theta(x^{(i)}) - y^{(i)}) \rightarrow$ Error, matlab prediction kitna galat hai.
- Is error ko square (²) kar dete hain — taaki negative aur positive errors cancel na ho jayein, aur bade errors ko zyada penalize kiya ja sake.
- Σ (Sigma) $\rightarrow$ Saare m data points ke squared errors ko add karna.
- $1/2m \rightarrow$ Average nikalne ke liye 'm' se divide karte hain, aur '2' se divide isliye karte hain taaki aage jab hum derivative (calculus) nikalenge, toh calculation simple ho jaaye (square ka power 2 cancel ho jaata hai).

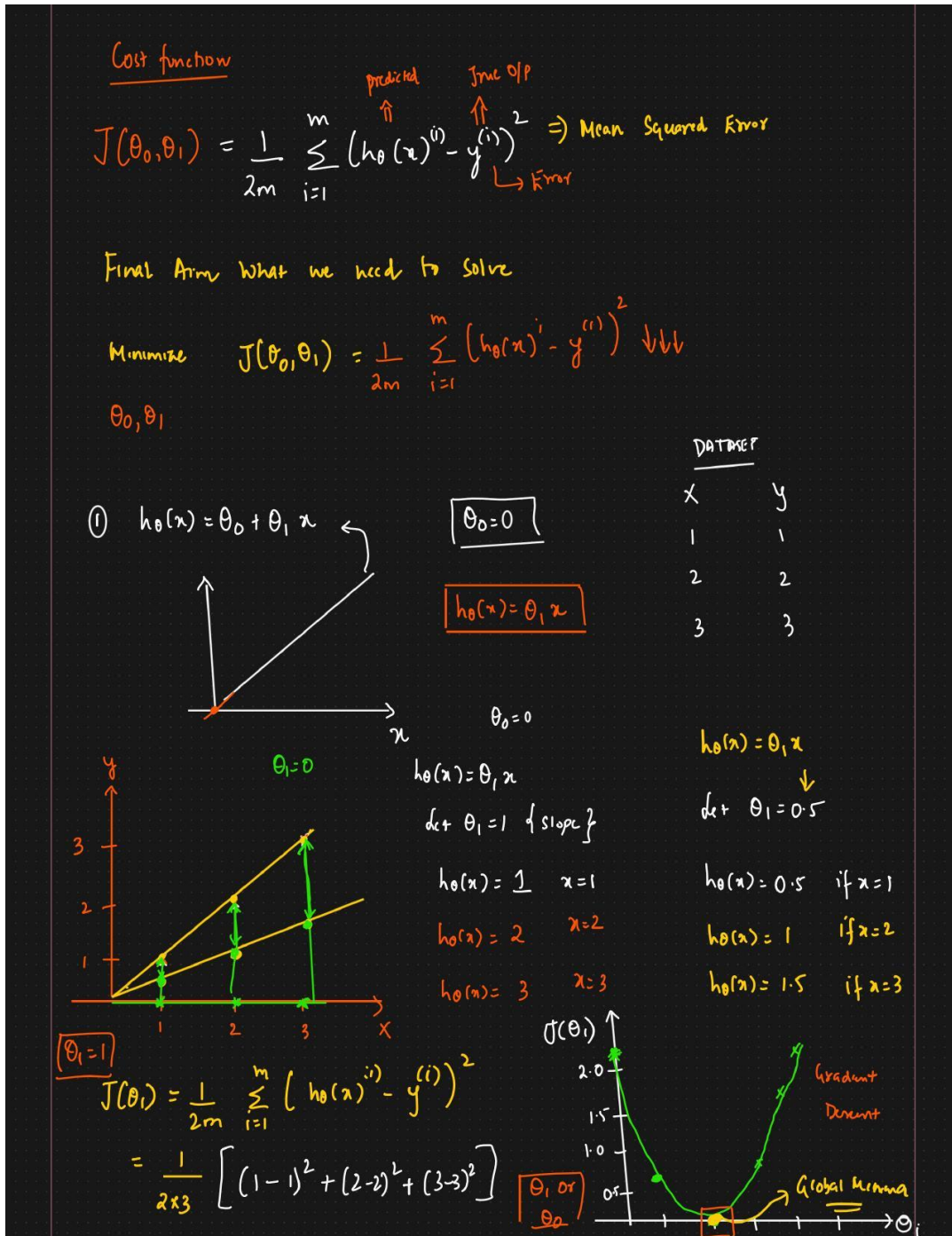
Isi wajah se is function ko Mean Squared Error (MSE) bhi kehte hain.

Final Goal / Aim

Hamara poora aim yeh hai ki hum aise θ_0 aur θ_1 dhoondein jinke liye $J(\theta_0, \theta_1)$ ki value sabse kam (Minimum) ho:

$$\text{Minimize } J(\theta_0, \theta_1) \text{ over } \theta_0, \theta_1$$

Jitna kam Cost Function ka value hoga, hamari Best Fit Line utni hi accurate hogi.



Isme dikhaya gaya hai: Cost Function $J(\theta_0, \theta_1)$ ki definition (Mean Squared Error), Minimize karne ka goal, aur $\theta_0=0$ fix karke different θ_1 values (0, 1, 0.5) ke saath worked example.

7. Ek Simple Example Se Samajhna ($\theta_0 = 0$ fix karke)

Concept ko aasan banane ke liye, hum abhi ke liye $\theta_0 = 0$ fix kar dete hain (yaani line origin se guzregi). Isse hamari equation simplify ho jaati hai:

$$h\theta(x) = \theta_1 x$$

Ab hum ek chhota dataset lete hain jahan X aur Y bilkul equal hain:

X	Y
1	1
2	2
3	3

Ab hum different θ_1 values try karke dekhte hain ki Cost J badalta kaise hai.

Case 1: $\theta_1 = 1$

Yahan slope = 1 le rahe hain, toh:

- $x = 1 \rightarrow h\theta(x) = 1$
- $x = 2 \rightarrow h\theta(x) = 2$
- $x = 3 \rightarrow h\theta(x) = 3$

Dekha jaaye toh predicted values (1, 2, 3) actual Y values (1, 2, 3) se bilkul match kar rahi hain — matlab yahan Error zero hai. Isliye:

$$J(\theta_1=1) = (1/(2 \times 3)) \times [(1-1)^2 + (2-2)^2 + (3-3)^2] = 0$$

Yeh sabse best case hai — Error puri tarah minimize ho chuka hai.

Case 2: $\theta_1 = 0.5$

- $x = 1 \rightarrow h\theta(x) = 0.5$
- $x = 2 \rightarrow h\theta(x) = 1$
- $x = 3 \rightarrow h\theta(x) = 1.5$

Ab predicted values, actual values se thodi door hain, isliye Error zero nahi hoga:

$$J(\theta_1=0.5) = (1/6) \times [(0.5-1)^2 + (1-2)^2 + (1.5-3)^2] \approx 0.58$$

Case 3: $\theta_1 = 0$

Agar $\theta_1 = 0$ rakhein, toh line bilkul flat (X-axis ke parallel) ho jaayegi, matlab har x ke liye $h\theta(x) = 0$.

$$J(\theta_1=0) = (1/6) \times [(0-1)^2 + (0-2)^2 + (0-3)^2] \approx 2.3$$

Yahan Error sabse zyada hai, kyunki predicted values actual values se kaafi door hain.

In Teeno Cases Se Kya Pata Chalta Hai?

Jab hum $J(\theta_1)$ ki values ko θ_1 ke against plot karte hain ($\theta_1 = 0, 0.5, 1, 1.5, 2, 2.5 \dots$), toh ek U-shape (bowl-shape / convex) curve milta hai. Is curve ka sabse neeche wala point — jahan J ki value sabse kam hai — usse Global Minima kehte hain. Notes ke example mein yeh minima $\theta_1 = 1$ par hai.

Note: Yahi wo point hai jahan hamari line data ko sabse achhe se fit karti hai. Isi minima ko dhoondhne ke liye ek algorithm use hota hai jise Gradient Descent kehte hain.

8. Gradient Descent — Convergence Algorithm

Gradient Descent ek optimization algorithm hai jiska kaam hai Cost Function $J(\theta_0, \theta_1)$ ko minimize karna, matlab θ_0 aur θ_1 ki aisi values dhoondhna jahan J sabse kam ho — bina saari possible values ko manually try kiye (jo ki bahut inefficient hoga).

Iska core idea yeh hai ki hum θ ki values ko baar-baar chhote-chhote steps mein update karte rehte hain, jab tak J ki value minimum (convergence) tak nahi pahunch jaati.

Repeat until convergence: $\{ \theta_j := \theta_j - \alpha \times (\partial J(\theta_j) / \partial \theta_j) \}$

Is formula ke har part ko samajhte hain:

- θ_j — yahan $j = 0$ ya 1 ho sakta hai, matlab yeh formula θ_0 aur θ_1 dono ke liye simultaneously apply hota hai.
- $:=$ — yeh 'assignment' symbol hai, matlab θ_j ki purani value ko naye calculated value se replace kar dena.
- α (alpha) — Learning Rate: yeh ek chhota positive number hota hai (jaise 0.001) jo yeh decide karta hai ki hum har step mein kitna bada 'jump' lein. Bahut chhota α ho toh convergence bahut slow hogi, aur bahut bada α ho toh hum minima ko 'miss' kar sakte hain.
- $\partial J(\theta_j) / \partial \theta_j$ — yeh Partial Derivative hai, jo J curve ke us point par ka Slope (Gradient) batata hai.

Slope (Derivative) Se Direction Kaise Milti Hai?

Yahi is algorithm ka sabse important intuition hai — Derivative humein batata hai ki curve ke us point par slope kis taraf 'tedha' hai, aur uske basis par θ ki value update hoti hai:

- Agar slope Negative hai (curve neeche se upar left side mein ja raha hai) $\rightarrow \theta_j - \alpha \times (\text{negative}) = \theta_j + (\text{positive})$ $\rightarrow$ yani θ_j ki value badh jaayegi (right ki taraf move karegi, minima ki taraf).
- Agar slope Positive hai (curve right side mein upar ja raha hai) $\rightarrow \theta_j - \alpha \times (\text{positive})$ $\rightarrow$ yani θ_j ki value ghat jaayegi (left ki taraf move karegi, minima ki taraf).

Is tarah, chahe hum minima ke left side se shuru karein ya right side se, Gradient Descent hamesha θ ko Global Minima ki taraf hi move karta hai — jab tak slope zero (flat) na ho jaaye, yani jab tak hum minima tak na pahunch jaayein.

$J(\theta_1) = 0 \leftarrow$
 $\theta_1 = 0.5$

Error has been 0.5 1 1.5 2.0 2.5
 minimized

$J(\theta_1) = \frac{1}{2 \times 3} \left[(0.5-1)^2 + (1-2)^2 + (1.5-3)^2 \right]$

$J(\theta_1) = \approx 0.58$
 if $\theta_1 = 0$

$J(\theta_1) = \frac{1}{2 \times 3} \left[(0-1)^2 + (0-2)^2 + (0-3)^2 \right]$

$J(\theta_1) \approx 2.3$

Convergence Algorithm {Optimize the changes of θ_1 value}

Repeat until convergence

θ_j value reach more efficiently

$\theta_j := \theta_j - \alpha \left[\frac{\partial J(\theta_j)}{\partial \theta_j} \right] \rightarrow -ve$

$\theta_j = \theta_j - \alpha (+ve)$
 $= \theta_j - (+ve)$

$\alpha = \text{Learning Rate}$

$\alpha = 0.001 \leftarrow$

$J(\theta)$

Derivate = slope

-ve slope

+ve slope

global Minima

Isme dikhaya gaya hai: $J(\theta_1)=0$ ki calculation ($\theta_1=0.5$ aur $\theta_1=0$ ke liye Error), aur Convergence Algorithm ka diagram jisme dikhaya gaya hai ki negative/positive slope ke basis par θ_j kaise update hota hai Global Minima ki taraf.

9. Partial Derivatives Ki Derivation (Calculus Part)

Ab hum actually calculate karte hain ki $\partial J(\theta_0, \theta_1) / \partial \theta_j$, θ_0 aur θ_1 ke liye kya banta hai. Iske liye basic calculus rule yaad rakhna hoga:

$$\partial(x^n) / \partial x = n \cdot x^{n-1} \Rightarrow \partial(x^2) / \partial x = 2x$$

Hamara Cost Function hai:

$$J(\theta_0, \theta_1) = (1/2m) \times \sum [h\theta(x^{(i)}) - y^{(i)}]^2$$

Case $j = 0$ (θ_0 ke liye derivative)

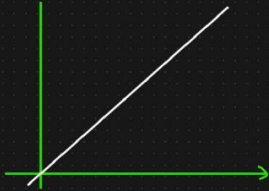
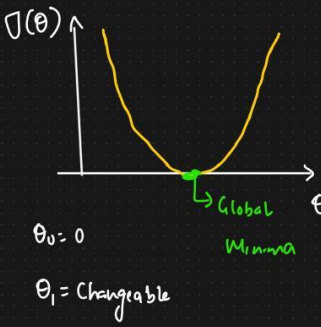
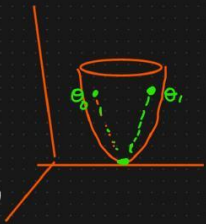
$h\theta(x) = \theta_0 + \theta_1 x$ mein, jab hum θ_0 ke respect mein derivative lete hain, toh $\theta_1 x$ wala term constant maana jaata hai (kyunki hum θ_0 ko differentiate kar rahe hain).

$$\partial J / \partial \theta_0 = (1/2m) \times \sum [2 (h\theta(x^{(i)}) - y^{(i)}) \times 1] = (1/m) \times \sum (h\theta(x^{(i)}) - y^{(i)})$$

Power rule ki wajah se square ka '2' bahar aata hai, aur woh denominator ke '2m' wale 2 ko cancel kar deta hai — isi wajah se cost function ki definition mein shuru mein '2' rakha gaya tha, taaki yeh calculation simple ho jaaye. Aakhri ' $\times 1$ ' isliye aaya kyunki θ_0 ka coefficient 1 hai (θ_0 ke saath koi x nahi multiply hota).

Final Conclusion

GRADIENT DESCENT

Convergence Algorithm

repeat until convergence

{

$\theta_j := \theta_j - \alpha \frac{\partial J(\theta_0, \theta_1)}{\partial \theta_j}$

}

$J(\theta_0, \theta_1) = \frac{1}{2m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)})^2$

$j = 0 \text{ and } 1$

$\frac{\partial}{\partial x} x^2 = 2x$

$\frac{\partial}{\partial x} x^h = x^h x^{h-1}$

$\rightarrow \frac{\partial}{\partial \theta_j} J(\theta_0, \theta_1) = \frac{\partial}{\partial \theta_j} \frac{1}{2m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)})^2$

if

$h_{\theta}(x) = \theta_0 + \theta_1 x \rightarrow 0$

$j=0 \Rightarrow \frac{\partial}{\partial \theta_0} J(\theta_0, \theta_1) = \frac{\partial}{\partial \theta_0} \frac{1}{2m} \left[\sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)})^2 \right]$

$= \frac{1}{m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)}) \times 1$

Isme dikhaya gaya hai: Final Conclusion diagram (Gradient Descent bowl-shape), Convergence Algorithm ka formula, aur $j=0$ (θ_0) ke liye partial derivative ki step-by-step calculation.

Case $j = 1$ (θ_1 ke liye derivative)

Ab jab hum θ_1 ke respect mein derivative lete hain, toh dhyaan dein ki $h_{\theta}(x) = \theta_0 + \theta_1 x$ mein θ_1 ke saath x multiply hai. Isliye chain rule apply hote hue x bahar aayega:

$$\partial(\theta_0 + \theta_1 x)/\partial \theta_1 = x$$

Isi wajah se puri derivative ban jaati hai:

$$\partial J/\partial \theta_1 = (1/2m) \times \Sigma [2 (h\theta(x^{(i)}) - y^{(i)}) \times x^{(i)}] = (1/m) \times \Sigma (h\theta(x^{(i)}) - y^{(i)}) \times x^{(i)}$$

Note: Dhyaan dijiye — θ_1 wali derivative mein extra ' $x^{(i)}$ ' multiply hota hai (θ_0 wali mein nahi), kyunki $h\theta(x)$ mein θ_1 hamesha x ke saath multiplied hota hai jabki θ_0 akela hota hai.

10. Final Gradient Descent Update Rules

Upar nikaali gayi dono derivatives ko wapas Gradient Descent ke formula mein daal dete hain. Yeh raha final, ready-to-implement algorithm:

Repeat until convergence: {

$$\theta_0 := \theta_0 - \alpha \times (1/m) \times \Sigma (h\theta(x^{(i)}) - y^{(i)})$$

$$\theta_1 := \theta_1 - \alpha \times (1/m) \times \Sigma (h\theta(x^{(i)}) - y^{(i)}) \times x^{(i)}$$

}

Important baat yeh hai ki θ_0 aur θ_1 dono ko Simultaneously (ek hi samay par) update kiya jaata hai — matlab dono naye values pehle temporarily calculate ki jaati hain, aur fir dono ko ek saath assign kiya jaata hai, purani values ka use karte hue hi dusri ki calculation honi chahiye.

Yeh process baar-baar repeat hota hai — har iteration mein θ_0 aur θ_1 thoda thoda update hote hain aur Cost Function J ki value chhoti hoti jaati hai — jab tak J ki value ek stable minimum point (Global Minima) tak converge nahi ho jaati. Wahi final θ_0 aur θ_1 ki values hamari trained Best Fit Line define karti hain.

$$j=1 \Rightarrow \frac{\partial}{\partial \theta_1} J(\theta_0, \theta_1) = \frac{\partial}{\partial \theta_1} \left[\frac{1}{2m} \sum_{i=1}^m (\theta_0 + \theta_1 x^{(i)} - y^{(i)})^2 \right]$$

$$= \frac{1}{m} \sum_{i=1}^m ((\theta_0 + \theta_1 x) - y^{(i)}) (x)$$

$$\frac{\partial}{\partial \theta_1} [\theta_0 + \theta_1 x] \Rightarrow x$$

Repeat until Convergence

$$\left\{ \begin{array}{l} \theta_0 := \theta_0 - \alpha \frac{1}{m} \sum_{i=1}^m (h_{\theta}(x)^{(i)} - y^{(i)}) \\ \theta_1 := \theta_1 - \alpha \frac{1}{m} \sum_{i=1}^m (h_{\theta}(x)^{(i)} - y^{(i)}) x^{(i)} \end{array} \right\}$$

Isme dikhaya gaya hai: $j=1$ (θ_1) ke liye partial derivative ki calculation, aur final Gradient Descent update rules (θ_0 aur θ_1 dono ke liye).

11. Poori Process Ka Summary

- Step 1: Dataset lo (X = input feature, Y = output/label).
- Step 2: Ek Hypothesis Function define karo — $h_{\theta}(x) = \theta_0 + \theta_1 x$.

- Step 3: Ek Cost Function $J(\theta_0, \theta_1)$ banao jo model ki error measure kare (Mean Squared Error).
- Step 4: Gradient Descent algorithm use karke θ_0 aur θ_1 ko baar-baar update karo, taaki J minimize ho.
- Step 5: Jab J convergence tak pahunch jaaye (yani ab aur kam nahi ho raha), tab final θ_0 aur θ_1 hi hamari Best Fit Line define karte hain.
- Step 6: Ab is trained line ka use karke naye input X ke liye output Y predict kiya ja sakta hai.

In short: Simple Linear Regression ek straight line dhoondhne ki koshish karta hai jo data ko best fit kare, aur Gradient Descent us line ke parameters (θ_0, θ_1) ko dhoondhne ka mechanism hai jo total error ko minimum kar de.